SOFTWARE REQUIREMENTS SPECIFICATION · ISO/IEC/IEEE 29148 STRUCTURE

AI-Based System-Wide Real-Time Masking of Offensive Language and People on Android Phones

An Android app that hides bad language and images of people on the screen, in the apps people use every day, with all AI running on the phone.

Version	Date	Scope
3.0 (draft)	21 October 2026	Graduation: v1–v4 · v5 bonus · v6 future work
Team	**Status**	
Ahmed Mohammed Omer · Omer Abd Almonim · Momen Kamal	For supervisor review	

1. Introduction 2. Overall description 3. External interfaces 4. Functional requirements 5. Use cases 6. Non-functional requirements 7. Verification & traceability 8. Out of scope & limits 9. Risks 10. Status & plan 11. Document history

1. Introduction

1.1 Purpose

This document states what the app must do (functional requirements), how well it must do it (non-functional requirements), and how each requirement will be checked. It is the agreement between the team and the supervisor, and the reference for design, testing and the final report. The technical study and test results are in **`investigation-android-v1.md`** and **`p0-results-android-v1.md`**; the design is in **`app-design-v1.md`**.

1.2 Problem

Some people want to avoid bad language and images of people while they use their phone: in comments, chats, social apps, ads and news. Today's tools have four problems:

- Most work in **one place only** (one browser, one website).
- Tools that watch the whole screen show content **first** and cover it **after**. The eye understands a picture in as little as 13 ms[R6], so a late cover is too late.
- Some send screen content to a server, which harms privacy.

- Almost none filter **Arabic** text, which is full of dialect, spelling tricks and context-dependent words.

1.3 Product scope

The product is one Android app. It uses Android's accessibility service (the feature screen readers use) to read the screen and draw covers on top of other apps. Bad words are found with a word list plus our own small AI text model; people are found with an AI segmentation model. Everything runs on the phone; the app has no internet permission.

VERSION	HIDES	IN GRADUATION SCOPE?
v1	**Bad-language text** in standard Arabic and English (swearing, sexual words, insults, blasphemy): word list + our own small text model. Also builds the app shell: setup, covers, pause, peek, notification.	Yes
v2	**Text inside images** (memes, thumbnails, burned-in subtitles), with on-phone OCR	Yes
v3	**Big people** (taller than about 96 screen pixels)	Yes
v4	**Medium people** (about 32–96 pixels), checked in tiles	Yes
v5	**Small people** (under about 32 pixels), with our own fine-tuned model	Bonus
v6	**People in playing videos**	No: future work

Each version adds to the one before. The **self-control mode** (FR-25) is future work after graduation.

1.4 Definitions and abbreviations

TERM	MEANING IN THIS DOCUMENT
Accessibility service	A special Android app component the user turns on in Settings. It can read what is on the screen, take a picture of an app window (Android 14+), and draw on top of other apps.
Cover	A solid block (text) or solid / strong-pixelation shape (people) drawn over content
Cover layer (overlay)	One full-screen, transparent accessibility window where all covers are drawn. Touches pass through it.
Hide first (fail-closed)	New content stays covered until it has been checked. When unsure, the system hides.

Peek	The user deliberately uncovers one covered item (FR-23)
Self-control mode	An optional mode where the user makes it harder for themselves to pause or peek (FR-25)
Word list / language pack	List of bad words and phrases with a category each / word list + text model for one language
Context word	A list entry that is offensive only in some sentences (for example كلب "dog", حمار "donkey")
Normalization	Removing spelling tricks before matching, so "كلبببب" and "كلب" count as the same word
Window picture	`takeScreenshotOfWindow`: a picture of one app window, without our covers; at most one every 333 ms
OCR	Optical character recognition: reading text from a picture
Segmentation	The model returns each person's exact shape (mask), not only a box
Tiles	Checking the screen in a few parts so small people look bigger to the model
Quantization	Storing model numbers with fewer bits (fp16, int8) to make it smaller and faster
Teacher / student (distillation)	Large models label data on a laptop; a small model learns from those labels and runs on the phone
Recall	Of all bad words (or people), the share that was covered. Our most important number.
p95	95% of measurements are at or below this value
Frame	One screen refresh. The test phone runs at 90 Hz, so one frame ≈ 11 ms.
MoSCoW	Priority: Must / Should / Could / Won't (this time)
FR / NFR / UC	Functional requirement / non-functional requirement / use case

1.5 References

- [R1] ISO/IEC/IEEE 29148:2018, *Systems and software engineering — Life cycle processes — Requirements engineering*.
- [R2] Team documents: `app-design-v1.md`, `investigation-android-v1.md`, `p0-results-android-v1.md`, `research-methods.md`, `roadmap.md`, `asbuild.md`.

- [R3] Android developers: `AccessibilityService`, `takeScreenshotOfWindow`, `TYPE_ACCESSIBILITY_OVERLAY`, accessibility button (`FLAG_REQUEST_ACCESSIBILITY_BUTTON`).
- [R4] Google LiteRT (on-device inference); PaddleOCR PP-OCRv5; Ultralytics YOLO26-seg.
- [R5] J. Nielsen, *Usability Engineering*, 1993: response-time limits (0.1 s instant, 1 s flow of thought).
- [R6] M. C. Potter et al., "Detecting meaning in RSVP at 13 ms per picture," *Attention, Perception, & Psychophysics*, 2014.
- [R7] HaramBlur browser extension and Android app (competitor study, investigation §9).

1.6 How this document is organised

§2 describes the product and its environment. §3 lists the interfaces. §4 gives the functional requirements grouped by feature, each with a priority and an acceptance check. §5 shows the main use cases. §6 gives quality targets. §7 says how each requirement is verified and links requirements to versions and research questions. §8–§10 give limits, risks and status. Requirement IDs are stable: an ID is never reused, so older documents stay valid.

2. Overall description

2.1 Product perspective

A new, self-contained product. It does not replace another system. It sits between the apps on the screen and the user's eyes, using only public Android APIs (no root).

Other apps
WhatsApp, Facebook, Chrome, YouTube… share their text with Android; their windows can be pictured

Our app (one accessibility service)
text path: list + text model · image path: OCR + people model · one cover layer · settings

User
sets up, pauses, peeks; sees the screen with covers

Android
accessibility events, text + per-letter boxes, window pictures, overlay

Phone hardware
CPU and GPU for the models; no NPU help on the test phone

No server
no internet permission; nothing leaves the phone

2.2 Product functions (summary)

- **Set up** with clear consent, then turn on the accessibility service once.
- **Read** the text on screen; **clean** it; **match** it against the word list; send unclear messages to the **text model**; **cover** the bad words.
- **Picture** the app window; **read text inside images** (OCR) and **find people**; cover them.

- **Hide first** where checking is slow (images, context words).
- **Covers follow scrolling**; touches pass through them.
- **Control:** on/off, pause, skipped apps, categories, languages, cover style, **peek** at one item.
- **Status:** notification, Quick Settings tile, warning cards.
- Later: **self-control mode**.

2.3 How it works, simply

Text (v1): Android reports a screen change → the app reads the visible text → normalizes it → looks up words in the list → clear bad words are covered at once; context words are covered at once and the text model decides whether the cover stays (FR-24) → covers drawn on each word's screen box.

Text in images (v2) and people (v3–v5): the changed area is covered at once → the app takes a window picture (which does not include our covers) → OCR reads text in images and the people model finds each person → covers stay only on bad words and people.

Content changes	Covered while checked	Shown
scroll, new page, new app	images: about 0.3–0.7 s today	with every detected person and bad word hidden

Why plain text is not covered first: the app learns *what* the text says from the same event that tells it the text appeared, and the list check takes under 1 ms. Covering all new text first would make the screen flash on every scroll without hiding anything sooner. The only gap is Android's event delay (NFR-3). Context words are the exception (FR-24) because the model takes longer.

2.4 User classes

USER	WHO	NEEDS	VERSION
Everyday user	A person who chooses to install the filter on their own phone. Any age from teen up; not technical. Arabic or English UI.	Simple setup, covers that do not get in the way, quick pause, peek when needed	v1+
Committed user	An everyday user who wants help keeping their own promise not to look at bad content	Make pausing and peeking slower or need someone else's PIN	Future (FR-25)
Trusted helper	A friend or family member the committed user chooses; holds the PIN on the same phone. Never monitors: sees no history.	Set a PIN; enter it when asked	Future (FR-25)

It is a personal tool, not a parental-control or monitoring tool. It keeps no history of what was covered beyond a daily count.

2.5 Operating environment

ITEM	REQUIREMENT
Android version	Text features (v1): Android 8.0+ (per-letter text boxes). Picture features (v2–v5): Android 14+ (window pictures). Older phones are not supported for picture features.
Test phone	Xiaomi Redmi 12: MediaTek Helio G88 (2 fast + 6 slow cores), Mali-G52 GPU, 8 GB RAM, Android 15 / HyperOS 2.0, 90 Hz screen. A normal low-cost phone: if it works here, it works on most phones.
Internet	Not needed and not requested.
Install	APK over ADB during development and the demo; Google Play later (the team has a developer account).

2.6 Design and implementation constraints

#	CONSTRAINT	SOURCE
C-1	No root, no custom ROM: public Android APIs only.	Project goal
C-2	A window picture at most every 333 ms per window, so at most ~3 image checks per second. This is why video is v6.	Android
C-3	Normal "draw over apps" windows are limited to 80% opacity when touches pass through; only the accessibility overlay can be fully opaque.	Android
C-4	A touch-through overlay cannot receive taps. Peek (FR-23) must switch the layer to touchable for a short time.	Android
C-5	Apps installed from a file must have "restricted settings" allowed before accessibility can be turned on (Android 13+).	Android
C-6	A normal app cannot stop the user from turning off its accessibility service or uninstalling it. Self-control mode can only add friction.	Android
C-7	No internet permission. All models run on the phone with LiteRT; big models only for training (laptop / Colab).	Privacy decision
C-8	Kotlin, Jetpack Compose, one app module. The text filter is plain Kotlin with no Android code, so it is testable on the laptop.	Design
C-9	The people model (Ultralytics YOLO) is AGPL-3.0, so the app must be open source; RF-DETR (Apache-2.0) is the fallback.	License

C-10	Google ML Kit OCR does not support Arabic; PaddleOCR is used.	Tooling

2.7 User documentation

In-app only: setup screens (FR-1), a "try it" page with sample text, and an About & privacy page. The report and a demo video document the system for examiners.

2.8 Assumptions and dependencies

- The user turns on the service in accessibility settings and allows background running (Xiaomi: Autostart, no battery limit).
- "Bad language" means the entries of our reviewed list in four categories (swearing, sexual, insults, blasphemy), plus what our text model flags.
- "Person" means **any human figure: real, drawn, anime / cartoon, or in a game**. Accuracy targets are first measured on photos; drawn and anime figures are measured as a separate group (see risk K-9).
- **Hiding too much is acceptable; showing what the user asked to hide is not.** When unsure, hide.
- Apps share their text with Android; where an app does not give per-word boxes, the whole message is covered.

3. External interface requirements

3.1 User interfaces

INTERFACE	CONTENT
Setup (one step per screen)	Welcome · languages · what to hide · consent (what the service can see) · turn on access (with the restricted-settings steps) · Xiaomi battery steps · notifications · try it
Home	Big status (On / Paused until hh:mm / Off / Problem) with a switch; pause 5 / 15 / 60 min; active languages; today's covered count (a number only); warning cards
Settings	Languages · categories · cover style (word / whole message) and colour · skipped apps · peek on/off · About & privacy. Later: strictness, people, self-control mode
Covers	Solid, slightly rounded blocks 2–4 dp larger than the word; neutral grey by default; people: solid or strong pixelation, mask grown a little. No text on covers.
Peek	Accessibility button (nav bar or floating) → covers get a thin outline for 5 s → tap one cover to show that item
Notification	Silent. "Filter is on · Arabic · 12 covered today" [Pause 15 min] [Turn off] · "Paused until 14:30" [Resume] · "Filter stopped: tap to fix"
Quick Settings tile	Tap: on/off. Long-press: open the app. Subtitle: On / Paused 14:30 / Off

All screens: Arabic (right-to-left) and English, following the phone's language; Material 3; work with TalkBack and large fonts (NFR-16).

3.2 Hardware interfaces

Screen (draw covers), CPU and GPU (models via LiteRT: GPU for the people model, CPU for OCR and the text model, to be measured together in v2/v3). No camera, microphone, location or storage access.

3.3 Software interfaces

INTERFACE	USED FOR	VERSION
`AccessibilityService` events and node tree	Know when the screen changes; read visible text; get per-letter boxes	v1

`TYPE_ACCESSIBILITY_OVERLAY` window	Draw all covers, fully opaque, touches pass through	v1
Accessibility button (`FLAG_REQUEST_ACCESSIBILITY_BUTTON`)	Start peek mode	v1
`takeScreenshotOfWindow`	Window picture for OCR and people (no covers in it)	v2+
LiteRT (CPU, GPU delegate)	Run the text model, OCR and people model	v1+
DataStore	Settings and the daily count	v1
`TileService`, notifications	Quick tile, status notification	v1

3.4 Communications interfaces

None. The app does not declare the `INTERNET` permission, so Android itself blocks all network access.

4. Functional requirements

Priority (MoSCoW): **Must** = needed for that version · **Should** = planned, can slip · **Could** = if time allows · **Future** = after graduation. **Verify**: T = test, D = demonstration, I = inspection, A = analysis (§7). New = added in v3.0.

4.1 F1 · Setup and consent

The user learns what the app sees before giving it access. Stimulus: first launch. Response: the setup steps (§3.1) end with the service on and a "try it" page.

ID	REQUIREMENT	VER.	PRIORITY	ACCEPTANCE CHECK
FR-1	Before access is turned on, show a clear consent screen: what the service can read, that nothing is saved or sent, and that the app has no internet access. Guide the user through "allow restricted settings" when Android blocks it.	v1	Must	I + D: consent cannot be skipped; on the Redmi 12 (HyperOS) a file-installed APK can be turned on by following the steps.

4.2 F2 · Status and control

The user always knows whether the filter is working, and can stop it briefly.

ID	REQUIREMENT	VER.	PRIORITY	ACCEPTANCE CHECK
FR-2	Show a silent notification while the filter runs. If the filter stops (crash, killed by the system), the user must be able to see it: the notification changes to "stopped" or disappears. Android may leave a killed app's notification on screen (issue #15); a periodic health check is the fallback.	v1	Must	T: kill the process 10 times; each time the stopped state is visible within 15 min.
FR-4	Turn the filter on and off (home, notification, tile), and choose skipped apps. Covers are removed at once when a skipped app opens.	v1	Should	D: switch in all three places; open a skipped app → no covers.
FR-22 New	Pause for 5, 15 or 60 minutes. The filter resumes by itself at the end and checks the screen again at once. Covers are removed at once on pause.	v1	Must	T: pause 5 min → covers gone; after 5 min covers return within one check.

4.3 F3 · Covers and peek

Covers hide content without blocking the app. The user can deliberately uncover one item.

ID	REQUIREMENT	VER.	PRIORITY	ACCEPTANCE CHECK
FR-3	Covers move with the page while scrolling, so covered content stays covered and covers do not flicker. Touches pass through covers.	v1	Must	T: 30 scroll trials per app in 3 apps, filmed at 240 fps: no frame shows a covered word.
FR-23 New	**Peek.** Pressing the accessibility button starts peek mode for 5 s: covers become tappable (outlined). Tapping a cover shows **that item only** (one word, one message, or one person). It stays shown while it stays on screen; it is covered again when it leaves the screen or the app changes. Tapping outside a cover, or 5 s without a tap, ends peek mode. Shown items are remembered in memory only, never saved. Peek can be turned off in settings, and is off in self-control mode. While peek mode is on, taps do not reach the app below (C-4).	v1	Should	T: peek one word of three → only that word shown; others stay covered; scroll it away and back → covered again; no tap for 5 s → touches pass through again.

4.4 F4 · Bad-language text (v1)

Two layers: 1 the word list (instant, exact), 2 our own small text model (context). Stimulus: an accessibility event with new visible text. Response: covers on bad words within the gap of NFR-3.

ID	REQUIREMENT	VER.	PRIORITY	ACCEPTANCE CHECK
FR-5	Read the visible text of every app that shares its text with Android (social apps, chats, browsers, news), skipping input boxes, our own app and system UI.	v1	Must	T: log what the service sees in Chrome, WhatsApp, Facebook, X, YouTube, Instagram, TikTok; report which give text and per-letter boxes.
FR-6	Find bad words and phrases in standard Arabic and English, in four categories: swearing, sexual words, insults, blasphemy. Each list entry is marked *always* or *context*.	v1	Must	I: list reviewed by two people independently, Cohen's kappa reported; every entry has a category and a source.
FR-7	Catch spelling tricks: tashkeel, tatweel (ـ), alef / ya / ta-marbuta forms, repeated letters, look-alike symbols (@ → a, $ → s, 0 → o).	v1	Must	T: shared test-vector file passes in both the Kotlin tests and `build_ar.py --test`.
FR-8	Match whole words only (after normalization and light prefix / suffix stripping), so a normal word that contains a bad word is not covered.	v1	Must	T: unit cases such as "class" ≠ "ass"; Arabic prefix cases (ال، ب، و) match.
FR-9	Cover with a solid block. The user chooses: the bad word only, or the whole message. If the app gives no per-letter boxes, cover the whole message.	v1	Must	D: both modes in Chrome; fallback shown in an app without boxes.
FR-10	Turn each category on and off.	v1	Should	T: category off → its words not covered.
FR-11	Do not cover text the user is typing.	v1	Should	T: type a listed word in a chat box → not covered; after sending → covered.
FR-20	Our own small on-phone text model as layer ②: reads the whole message, returns a category with a confidence,	v1	Should	A + T: on a fixed test set, better than the word list (paired

	and marks the offensive			bootstrap) and meets

4.5 F5 · Text inside images (v2)

ID	REQUIREMENT	VER.	PRIORITY	ACCEPTANCE CHECK
FR-12	Read text inside images (memes, thumbnails, burned-in subtitles) from the window picture with on-phone OCR (PaddleOCR, Arabic and English mobile models), pass it through F4, and cover the bad words. Only picture areas are read; normal text already came through F4.	v2	Must	T: on a set of real screenshots, targets of NFR-14 (set after test P0d).
FR-24 New	**Hide first for context words.** When the list finds a *context* entry, cover it at once, then ask the text model. The cover is removed only if the model says the message is clean with confidence above the strictness threshold. If the	v1	Should	T: "حمار" in a story about animals → covered, then uncovered; as an insult → stays covered; model disabled → stays covered

4.6 F6 · People (v3–v5)

Stimulus: a screen change in an app that allows window pictures. Response: changed area covered at once; after the check, covers stay only on people.

ID	REQUIREMENT	VER.	PRIORITY	ACCEPTANCE CHECK
FR-13	Detect every person (real, drawn, anime, game) and cover their shape with a solid fill or strong pixelation, grown a little. A light blur is not allowed (it shows the body shape). All people are hidden; no gender guess.	v3	Must	T: NFR-4 on real screenshots; drawn / anime figures reported as a separate group.
FR-14	Hide first: cover changed content as soon as the change is reported and keep it covered until checked. A cover never lifts because of a timeout.	v3	Must	T: film at 240 fps; no frame shows an unchecked person.
FR-15	Apps that block screenshots (banking, Netflix, other protected video) cannot be pictured: their images are not checked; their normal text still is. Optional switch: "always cover this app"	v3	Could	D: Netflix → text filtered, image path skipped.

FR-16	Android 13-and-older fallback with screen capture. Dropped 18 Oct 2026.	—	Won't	—
FR-17	Check the changed area in tiles to find medium-size people.	v4	Must	T: NFR-4 (medium) and NFR-5 on real screenshots.
FR-18	Find small and far-away people with our own model fine-tuned on real phone screenshots (and drawn / anime pictures if K-9 shows a gap).	v5	Could (bonus)	T: beats the base model on small people on the same test set.

4.7 F7 · Video (v6, after graduation)

ID	REQUIREMENT	VER.	PRIORITY	ACCEPTANCE CHECK
FR-19	Hide people in playing videos, often enough to follow movement. Needs its own method (tracking between checks, or a faster picture source during video only), because of C-2.	v6	Future	—

4.8 F8 · Self-control mode (future)

For users who want to hold themselves to their own choice. It adds friction; it cannot be a lock (C-6). It never monitors or records what the user looks at.

ID	REQUIREMENT	VER.	PRIORITY	ACCEPTANCE CHECK
FR-25 New	An optional mode, off by default. When on: (a) pause and turn-off start a waiting time the user chose (for example 1–30 min) before they take effect, and can be cancelled; (b) peek (FR-23) is off; (c) leaving the mode needs a longer wait (for example 24 h) **or** a PIN set by a trusted helper on the same phone; (d) Home shows honestly that Android still lets anyone turn off accessibility or uninstall the app.	Future	Future	T: each of (a)–(c) on the phone.

5. Use cases

UC-1 · First setup

Actor	Everyday user
Pre	App installed; access off
Main flow	1. Opens the app. 2. Picks languages and categories. 3. Reads consent (FR-1). 4. Taps "Turn on access", enables the service in Settings. 5. App detects it and continues. 6. Xiaomi battery and notification steps. 7. "Try it" shows covered sample words.
Alternate	4a. Android blocks access (restricted settings): the app shows App info → ⋮ → "Allow restricted settings", then returns to step 4.
Post	Filter on; notification shown
Covers	FR-1, FR-2, FR-21

UC-2 · Read a chat with the filter on

Actor	Everyday user
Pre	Filter on; chat app shares its text
Main flow	1. New message appears. 2. Android sends an event; the service reads the text. 3. Normalize → match. 4. An *always* word is covered at once. 5. A *context* word is covered at once; the model checks the message; cover stays or is removed (FR-24). 6. User scrolls; covers move with the text.
Alternate	2a. App gives no per-letter boxes → whole message covered. 3a. User is typing → their input box is not covered.
Post	Bad words covered; clean text visible
Covers	FR-3, FR-5 – FR-11, FR-20, FR-24

UC-3 · Scroll a feed with pictures

Actor	Everyday user
Pre	Android 14+; v3 or later; app allows window pictures
Main flow	1. User scrolls; new posts appear. 2. Changed area covered at once (FR-14). 3. Window picture taken (no covers in it). 4. OCR reads text in images; people model finds people (tiles for medium people in v4). 5. Covers stay on people and bad words; the rest is uncovered (≤ 0.7 s, NFR-5).
Alternate	1a. User scrolls again before the check ends → old check dropped; only the latest screen is checked. 3a. App blocks pictures → image path skipped, text still filtered (FR-15).
Post	People and bad words in images covered
Covers	FR-12 – FR-15, FR-17

UC-4 · Peek at one covered item

Actor	Everyday user
Pre	Filter on; peek allowed; self-control mode off
Main flow	1. User presses the accessibility button. 2. Covers are outlined for 5 s. 3. User taps one cover. 4. That item is shown; peek mode ends. 5. When the item leaves the screen, it is covered again.
Alternate	3a. No tap for 5 s, or tap outside covers → peek mode ends, nothing shown.
Covers	FR-23

UC-5 · Pause and resume

Actor	Everyday user
Main flow	1. User taps "Pause 15 min" (notification, tile or home). 2. All covers removed. 3. After 15 min the filter resumes and checks the screen at once.
Alternate	1a. Self-control mode on (future) → a waiting time starts first (FR-25).
Covers	FR-22, FR-25

UC-6 · Turn on self-control mode (future)

Actors	Committed user, trusted helper
Main flow	1. User opens Settings → Self-control. 2. Chooses the pause waiting time. 3. Chooses "wait 24 h to leave" or "helper's PIN". 4. If PIN: the helper types it on the phone. 5. Mode on; peek disabled.
Covers	FR-25

6. Non-functional requirements

All phone measurements on the Redmi 12, with battery temperature and Android build recorded (research-methods §4.6).

6.1 Safety (how much is hidden)

ID	TARGET	HOW MEASURED	TODAY
NFR-1	Hide ≥ 95% of the listed bad words in real comments and chats.	OSACT tweets, English comments and our own screenshots	Not measured
NFR-2	≤ 1 wrong cover per 1 000 clean words.	Clean text: news, **Quran**, schoolbooks, normal chats	Not measured
NFR-4	Hide ≥ 95% of people of that version's size (big v3, medium v4, small v5). "Hidden" = ≥ 90% of the person under the cover.	Real phone screenshots, split by size; drawn / anime reported separately	Not yet Synthetic screens, one pass (tiles): big 85–89% (91–95%), medium 60–76% (87–89%), small 24–27% (51–54%).

6.2 Performance

ID	TARGET	WHY THIS NUMBER	TODAY
NFR-3	New text shows for ≤ 3 frames (≈ 33 ms at 90 Hz) before its cover appears.	Below the 0.1 s "instant" limit [R5]; filmed at 240 fps, ≥ 30 trials per app	Not measured First measurement in v1.
NFR-5	Changed images stay covered ≤ 0.7 s while checked.	Under the 1 s "flow of thought" limit [R5]. Covering first is needed because a picture can be understood in 13 ms [R6].	Not measured Expected 0.3–0.7 s from the parts.
NFR-6	One full-screen people check ≤ 150 ms.	So that tiles fit in NFR-5	Not met GPU ~205 ms (189 ms over 15 min non-stop); best CPU 277 ms. v4 tiles only changed areas.

NFR-13	Text model, per pack: real RAM ≤ 100 MB, ≤ 50 ms per message (p95), about one CPU core at most.	Runs beside the people model on a low-cost phone	Not built Candidates measured first (step P0c).
NFR-14	Text in images: recall and speed targets set after the OCR phone test.	No Arabic OCR phone numbers exist yet	Not set

6.3 Resources

ID	TARGET	TODAY
NFR-8	No work while the screen is still. Battery: 30 min of scrolling with the filter on vs off, difference reported; one full day in the v1–v4 test phase.	By design Battery not measured yet.
NFR-9	App memory: v1 < 50 MB; with text model < 150 MB; with people model < 250 MB.	Likely People model alone 87 MB (CPU) – 180 MB (GPU) peak.

6.4 Reliability

ID	TARGET	TODAY
NFR-11	A crash in our app never closes or freezes another app.	By design
NFR-15 New	**Fail toward hiding, visibly.** If a model fails to load, its layer falls back (text: list only; images: area stays covered) and a warning card appears. If the overlay cannot be drawn, the notification says "stopped".	By design

6.5 Security and privacy

ID	TARGET	TODAY
NFR-10	Screen text and pictures stay in memory only; nothing saved, nothing sent; no `INTERNET` permission; release builds never log screen text. Only a daily count is stored.	By design Verify: I of the manifest and logs.
NFR-12	The user grants access **once**; never asked again per use or after restart.	By design A top complaint about a competitor.

6.6 Usability and accessibility

ID	TARGET	TODAY
NFR-7	Touches pass through covers; scrolling does not feel slower (no extra dropped frames with the overlay on vs off).	By design Verify: frame stats, overlay on vs off.
NFR-16 New	Our own screens work with TalkBack (every control labelled), at 200% font size, and in right-to-left Arabic.	Planned
NFR-17 New	A first-time user finishes setup without help. Checked with at least 5 people outside the team.	Planned

6.7 Maintainability and legal

ID	TARGET	TODAY
NFR-18 New	The text filter has no Android code and its unit tests run on the laptop; one shared test-vector file keeps the Python list builder and the app's cleaning identical.	By design
NFR-19 New	Every word-list source, dataset and model has its license recorded; AGPL-3.0 is met by publishing the app's source; research-only data is not shipped; offensive data is not published beyond short examples.	Partly Open: Attia lexicon permission; AraBERT / OSACT authors.

8. Out of scope and known limits

ITEM	WHY
iPhone	Apple does not let apps see or cover other apps.
Arabic dialects and Arabizi	Possible after v1 as extra packs; the list already holds 936 Arabizi entries.
Root mode inside Android's screen drawing	Against C-1.
Linux desktop version	Paused as plan B; its test results still help.
Hiding only some people (for example by gender)	A wrong guess shows a person, and small, turned or drawn people cannot be judged. All people are hidden.
A real lock against turning the filter off	Android does not allow it (C-6). Self-control mode (FR-25, future) adds friction only.
Pictures of apps that block screenshots (banking, Netflix…)	Android does not allow it. Their text is still filtered.
Text in apps that do not share it (many games)	Only the picture checks (v2–v4) work there.
Video while it plays	v6, after graduation (C-2).
Android older than 14 for picture features	No window pictures; FR-16 dropped.
Accounts, cloud, statistics beyond a daily count, monitoring of others	Privacy; not a monitoring tool.

9. Risks

#	RISK	EFFECT	WHAT WE DO
K-1	No complete free Arabic list (best free one: 348 Arabic-script entries)	Arabic words missed	Combined 7 sources (1 906 terms); team review with kappa; test on real comments
K-2	Insults and blasphemy depend on context; words like كافر appear in the Quran	Clean text covered	*context* marking, text model (FR-20, FR-24), phrases, Quran in the NFR-2 test
K-3	Some apps do not share text or per-letter boxes	Missed words or bigger covers	Log 7 apps in v1-a; whole-message covers; OCR in v2
K-4	Small people are missed	Some people shown	Tiles, own training (v5), threshold "hide when unsure"
K-5	The phone is too slow; heat slows the CPU by up to ~30%	Longer waits, no tiles	GPU for people; tiles only on changed areas; plan with warm numbers
K-6	Xiaomi's battery saver stops the service	Filter stops silently	Setup step for Autostart; FR-2 stopped state
K-7	Competitor HaramBlur (~700K installs)	Must offer something new	Text (screen and images), hide first, cover only the person, permission once, all on the phone
K-8	Arabic OCR too slow or weak on the phone	v2 waits or misses words	Measure first (P0d); only changed images; smaller input
K-9 New	**Drawn and anime people.** The people model was trained on photos (COCO); drawings, anime and game characters look different (domain gap).	Drawn people missed more often	Add drawn / anime screenshots to the v3 test set and report them separately; if weaker, add drawn pictures to v5 training (licenses checked)
K-10 New	The accessibility button is hidden or placed differently on HyperOS	Users cannot find peek	Check on the Redmi 12 in v1-a; fallback: a "show this screen 10 s" notification action
K-11	Play Protect or Android 17 flags an app that reads the screen and draws over it	Install warnings	No internet, minimal permissions, clear consent, honest description; test early

10. Status and plan

STEP	WHAT	STATUS
`P0 / P0b`	People model: laptop and phone speed and accuracy; 15-minute sustained test	**Done** GPU ~205 ms per screen at 640 px
`Lists`	Combined Arabic list (1 906 terms, 7 sources)	**Done** Review pending
`v1-a`	App skeleton: service, cover layer, settings, notification, consent, pause, peek	**Next**
`v1-b / c`	Text filter and gap measurement; list review	**Planned**
`v1 model`	Own text model: candidates on the phone (P0c), teacher labels, training, comparison	**Planned**
`P0d / v2`	Arabic OCR phone test; text in images	**Planned**
`v3 / v4`	Big people; tiles for medium people	**Planned**
`Test`	v1–v4 together, real screenshots, a full day on battery	**Planned**
`v5`	Own model for small people	**Bonus**
`v6, FR-25`	Video; self-control mode	**After graduation**

Plan by week: **`roadmap.md`**. Deadline: about mid-April 2027.

11. Document history

VERSION	DATE	CHANGE
1.0	2026-10-03	Linux desktop version (now paused: `srs-linux-en.html`)
2.0–2.2	2026-10-06 – 18	Android; text first; text in images and video added; six-month plan
2.3	2026-10-18	Text model merged into v1; graduation scope v1–v4; honest limits
3.0	2026-10-21	Rewritten in ISO/IEC/IEEE 29148 structure: scope, definitions, references, user classes, constraints, interfaces, acceptance check per requirement, use cases, NFRs by category, verification methods, traceability to research questions. New: pause (FR-22), peek (FR-23), hide first for context words (FR-24), self-control mode (FR-25, future), NFR-15 – NFR-19, risks K-9 (drawn / anime people) and K-10. FR-2 now accepts a "stopped" notification.